

Ejection Chains en Grafos Bipartitos construidos mediante GRASP

*Formulaciones, Dominios y Métodos de Búsqueda
con Conjunto Eyectado Mínimo*

Informe Técnico

April 6, 2026

Contents

1	Introducción y Motivación	2
2	Preliminares y Notación	2
3	Formulación del Dominio en la Ejection Chain	3
3.1	Dominio del nodo inicial	3
3.2	Dominio del nodo eyectado en el paso k	3
3.3	Condición de subconjunto del dominio inicial	3
4	Modelado como Problema de Camino Más Corto	4
4.1	Estructura del estado y reconstrucción	4
5	Estrategias de Restricción del Dominio	5
5.1	Nivel 1 — Filtro estructural (condición de subconjunto)	5
5.2	Nivel 2 — Filtro de calidad (GRASP-guided)	5
5.3	Nivel 3 — Filtro anti-ciclo	5
6	Algoritmos de Búsqueda	6
6.1	BFS — Búsqueda en Anchura (cadena de longitud mínima garantizada)	6
6.2	A* — Búsqueda Heurística (cadena óptima con menor exploración) . .	7
7	Comparación de Métodos	9
8	Procedimiento Completo	10
9	Ejemplo Ilustrativo	10
10	Recomendaciones de Implementación	11
11	Conclusiones	11

1 Introducción y Motivación

Las **Ejection Chains** (cadenas de eyección) son una técnica de búsqueda local avanzada introducida por Glover (1991) en el contexto de la metaheurística Tabu Search. Su principio central consiste en realizar una secuencia de reasignaciones encadenadas, donde cada movimiento “eyecta” un elemento de su posición actual y lo desplaza a otra, de manera que el elemento desplazado queda libre para iniciar el siguiente eslabón de la cadena.

Cuando la solución inicial proviene de un procedimiento **GRASP** (Greedy Randomized Adaptive Search Procedure), el grafo bipartito resultante posee una estructura bien definida que permite explotar sistemáticamente los dominios de los nodos para construir cadenas cortas y de alta calidad.

Este informe presenta:

- la formulación matemática del problema de dominios en grafos bipartitos,
- la condición de subconjunto $D_k(x_k) \subseteq D(x_0)$,
- el modelado de la búsqueda de cadena mínima como un problema de camino más corto, y
- los algoritmos BFS y A* con heurística admisible, junto con estrategias de poda de dominio.

2 Preliminares y Notación

Definición 2.1 (Grafo Bipartito con Asignación). Sea $G = (X \cup Y, E)$ un grafo bipartito donde X es el conjunto de *agentes* e Y el conjunto de *recursos*. Una *asignación factible* es una función inyectiva $f : X \rightarrow Y$ tal que $(x, f(x)) \in E$ para todo $x \in X$.

Definición 2.2 (Vecindad y Dominio). Para un nodo $x \in X$, su *vecindad* en G es $N(x) = \{y \in Y \mid (x, y) \in E\}$. El *dominio de reasignación* de x dado el estado actual S es:

$$D(x) = N(x) \setminus \{f(x)\},$$

es decir, los recursos alcanzables por x distintos de su asignación vigente.

Definición 2.3 (Nodo Eyectado). Un nodo $x_k \in X$ queda *eyectado* en el paso k de la cadena cuando el nodo anterior x_{k-1} se reasigna al recurso $y_{k-1} = f(x_k)$, privando a x_k de su asignación.

3 Formulación del Dominio en la Ejection Chain

3.1 Dominio del nodo inicial

Sea x_0 el nodo que se desea mover (nodo inicial de la cadena). Su dominio en el paso 0 es:

$$D_0(x_0) = D(x_0) = N(x_0) \setminus \{f(x_0)\}. \quad (1)$$

3.2 Dominio del nodo eyectado en el paso k

Cuando x_{k-1} elige $y_{k-1} \in D_{k-1}(x_{k-1})$, el nodo $x_k = f^{-1}(y_{k-1})$ es eyectado. Su dominio candidato en el paso k es:

$$D_k(x_k) = (N(x_k) \setminus \{f(x_k)\}) \setminus \mathcal{V}_{k-1}, \quad (2)$$

donde $\mathcal{V}_{k-1} = \{y_0, y_1, \dots, y_{k-2}\}$ es el conjunto de recursos ya visitados en la cadena (filtro anti-ciclo).

3.3 Condición de subconjunto del dominio inicial

La condición central para que la cadena sea *válida* respecto al dominio inicial establece:

$$\boxed{D_k(x_k) \subseteq D_0(x_0)} \quad \forall k \geq 1. \quad (3)$$

Esta condición puede imponerse mediante tres mecanismos complementarios:

Filtro directo. En cada paso k , antes de extender la cadena:

$$D_{k,\text{filt}}(x_k) = D_k(x_k) \cap D_0(x_0).$$

Etiquetado de Glover. Asignar a cada recurso $y \in Y$ una etiqueta de nivel:

$$\ell(y) = \begin{cases} 0 & \text{si } y \in D_0(x_0), \\ k & \text{si } y \text{ fue liberado en el paso } k > 0. \end{cases}$$

Solo se permiten extensiones a recursos con $\ell(y) = 0$.

Condición estructural. Si el grafo satisface $N(x_k) \subseteq N(x_0)$ para todo x_k en la cadena (vecindades anidadas), la condición (3) se cumple automáticamente.

4 Modelado como Problema de Camino Más Corto

El objetivo de mover $x_0 \rightarrow y^*$ con el **número mínimo de eyecciones** se reduce a encontrar el camino más corto en el *grafo de eyección* G_E .

Definición 4.1 (Grafo de Eyección G_E). El grafo de eyección $G_E = (V_E, A_E)$ asociado a la solución S se define por:

$$V_E = \{(x, y) \mid x \in X, y \in D(x)\}, \quad (4)$$

$$A_E = \{(x_i, y_j) \rightarrow (x_k, y_\ell) \mid x_k = f^{-1}(y_j), y_\ell \in D(x_k)\}, \quad (5)$$

con función de costo $c((x_i, y_j), (x_k, y_\ell)) = 1$ para minimizar el número de nodos eyectados.

Proposición 4.1. *Un camino de longitud k en G_E desde (x_0, y^*) hasta cualquier estado (x_k, y_{libre}) con $y_{\text{libre}} \notin \text{Im}(f)$ corresponde a una ejection chain que mueve $x_0 \rightarrow y^*$ eyectando exactamente k nodos intermedios.*

4.1 Estructura del estado y reconstrucción

Un estado (x_k, y_k) en G_E codifica: “el nodo x_k desea ser asignado al recurso y_k ”. La cadena completa se representa como:

$$x_0 \xrightarrow{y_0} x_1 \xrightarrow{y_1} \dots \xrightarrow{y_{k-1}} x_k \xrightarrow{y^*} \emptyset,$$

donde $y^* \notin \text{Im}(f)$ es un recurso libre.

La **reconstrucción** de la solución se realiza en orden inverso:

1. $f(x_k) \leftarrow y^*$
2. $f(x_{k-1}) \leftarrow y_{k-1}$ (ahora libre)
3. $\vdots$
4. $f(x_0) \leftarrow y_0$

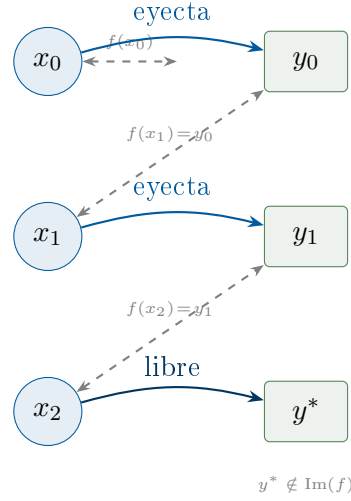


Figure 1: Estructura de una ejection chain de longitud 2: x_0 eyecta a x_1 , que a su vez eyecta a x_2 , el cual se reasigna a un recurso libre.

5 Estrategias de Restricción del Dominio

Para mantener cadenas cortas se aplican tres niveles de poda sobre $D(x_k)$:

5.1 Nivel 1 — Filtro estructural (condición de subconjunto)

$$D_{k,(1)}(x_k) = D_k(x_k) \cap D_0(x_0). \quad (6)$$

Elimina recursos que no pertenecen al espacio de soluciones alcanzable desde x_0 .

5.2 Nivel 2 — Filtro de calidad (GRASP-guided)

$$D_{k,(2)}(x_k) = \left\{ y \in D_{k,(1)}(x_k) \mid \Delta f(x_k \rightarrow y) \geq \theta \right\}, \quad (7)$$

donde θ es el umbral mínimo de variación en la función objetivo y $\Delta f(x_k \rightarrow y) = f_{\text{obj}}(S') - f_{\text{obj}}(S)$ con S' obtenida al reasignar $x_k \rightarrow y$.

5.3 Nivel 3 — Filtro anti-ciclo

$$D_{k,(3)}(x_k) = D_{k,(2)}(x_k) \setminus \mathcal{V}_{k-1}, \quad \mathcal{V}_{k-1} = \{y_0, \dots, y_{k-2}\}. \quad (8)$$

Garantiza que la cadena no revisita recursos ya procesados.

Observación 5.1. La composición de los tres filtros da el *dominio efectivo*:

$$D_{k,\text{eff}}(x_k) = (D(x_k) \cap D(x_0)) \setminus (\mathcal{V}_{k-1} \cup \{f(x_k)\}).$$

6 Algoritmos de Búsqueda

6.1 BFS — Búsqueda en Anchura (cadena de longitud mínima garantizada)

BFS es el algoritmo de referencia cuando el costo de arco es uniforme ($c = 1$), ya que garantiza encontrar la cadena con el menor número de eyecciones.

Algorithm 1 BFS para Ejection Chain Mínima

Require: f , $D_{k,\text{eff}}(\cdot)$, x_0 , y^*

Ensure: Camino mínimo π o Ninguno

```

1: inicio  $\leftarrow (x_0, y^*)$ 
2:  $Q \leftarrow \langle (\text{inicio}, [\text{inicio}]) \rangle$ 
3: visitado  $\leftarrow \{\text{inicio}\}$ 
4: while  $Q \neq \emptyset$  do
5:    $((x, y), \pi) \leftarrow Q.\text{pop\_izquierda}()$ 
6:   if  $y \notin \text{Im}(f)$  then return  $\pi$  ▷ Recurso libre: cadena válida
7:   end if
8:    $x' \leftarrow f^{-1}(y)$ 
9:   para cada  $y' \in D_{k,\text{eff}}(x')$  hacer
10:     $s' \leftarrow (x', y')$ 
11:    if  $s' \notin \text{visitado}$  then
12:      visitado  $\leftarrow \text{visitado} \cup \{s'\}$ 
13:       $Q.\text{encolar}((s', \pi + [s']))$ 
14:    end if
15:  end for
16: end while return Ninguno

```

```

1  from collections import deque
2
3  def bfs_ejection(assignment, eff_domain, source, target):
4      """
5          assignment    : dict x -> y
6          eff_domain    : callable (x, visited) -> set de recursos
7                        candidatos
8          source        : nodo inicial x_0
9          target        : recurso destino y*
10         """
11         inv = {v: k for k, v in assignment.items()}
12         free = set(assignment.values()) # Im(f)
13
14         start = (source, target)
15         queue = deque([(start, [start], {target})]) # estado, camino,
16                 visitados
17         seen = {start}

```

```

17     while queue:
18         (node, slot), path, visited = queue.popleft()
19
20         if slot not in free:                # recurso libre -> éxito
21             return path
22
23         evicted = inv[slot]
24         for nxt in eff_domain(evicted, visited):
25             state = (evicted, nxt)
26             if state not in seen:
27                 seen.add(state)
28                 queue.append((state, path + [state], visited | {nxt}
29                               )))
30
31     return None

```

Listing 1: Implementación BFS en Python

6.2 A* — Búsqueda Heurística (cadena óptima con menor exploración)

A* reduce el espacio explorado mediante una función heurística admisible $h : V_E \rightarrow \mathbb{R}_{\geq 0}$.

Definición 6.1 (Heurística Admisible para Ejection Chains). Sea $Y_{\text{libre}} = Y \setminus \text{Im}(f)$ el conjunto de recursos libres. Se define:

$$h(x, y) = \begin{cases} 0 & \text{si } y \in Y_{\text{libre}}, \\ 1 & \text{si } \exists y' \in D(f^{-1}(y)) \cap Y_{\text{libre}}, \\ 2 & \text{en otro caso (cota conservadora).} \end{cases}$$

Teorema 6.1. *La heurística de la Definición 6.1 es admisible (no sobreestima el número real de pasos restantes) y, por lo tanto, A* con esta heurística devuelve una ejection chain de longitud mínima.*

Esquema. Si $y \in Y_{\text{libre}}$, el estado ya es meta: $h = 0 \leq 0$ pasos reales. Si el ocupante de y tiene un recurso libre en su dominio, se requiere exactamente 1 eyección más: $h = 1$. En caso contrario, se necesitan al menos 2 pasos adicionales: $h = 2$. En ningún caso h supera el número real de eyecciones restantes. $\square$ $\square$

Algorithm 2 A* para Ejection Chain Mínima**Require:** $f, D_{k,\text{eff}}(\cdot), h, x_0, y^*$ **Ensure:** Camino óptimo π o Ninguno

```

1:  $s_0 \leftarrow (x_0, y^*)$ 
2:  $\text{montículo} \leftarrow [(h(s_0), 0, s_0, [s_0])]$ 
3:  $\text{mejor} \leftarrow \{\}$ 
4: while  $\text{montículo} \neq \emptyset$  do
5:    $(f, g, (x, y), \pi) \leftarrow \text{extraer\_mín}(\text{montículo})$ 
6:   if  $y \in Y_{\text{libre}}$  then return  $\pi$ 
7:   end if
8:   if  $\text{mejor}[(x, y)] \leq g$  then
9:     continuar
10:  end if
11:   $\text{mejor}[(x, y)] \leftarrow g$ 
12:   $x' \leftarrow f^{-1}(y)$ 
13:  para cada  $y' \in D_{k,\text{eff}}(x')$  hacer
14:     $g' \leftarrow g + 1; \quad s' \leftarrow (x', y'); \quad f' \leftarrow g' + h(s')$ 
15:     $\text{insertar}(\text{montículo}, (f', g', s', \pi + [s']))$ 
16:  end for
17: end while return Ninguno

```

```

1  import heapq
2
3  def heuristic(slot, inv, domains, free):
4      if slot in free:
5          return 0
6      occ = inv.get(slot)
7      if occ and any(s in free for s in domains[occ]):
8          return 1
9      return 2
10
11 def astar_ejection(assignment, eff_domain, source, target):
12     inv = {v: k for k, v in assignment.items()}
13     free = set(y for y in assignment.values()
14                if y not in assignment) # recursos sin dueño
15     # recalcular correctamente
16     all_resources = set(assignment.values())
17     # free slots = los que no estan en Im(f)
18     # (en la practica se pasan como parametro)
19
20     s0 = (source, target)
21     h0 = heuristic(target, inv, eff_domain, free)
22     heap = [(h0, 0, s0, [s0])]
23     best = {}
24
25     while heap:

```

```

26         f, g, (node, slot), path = heapq.heappop(heap)
27
28         if slot not in all_resources:      # recurso libre
29             return path
30
31         if best.get((node, slot), float('inf')) <= g:
32             continue
33         best[(node, slot)] = g
34
35         evicted = inv.get(slot)
36         if evicted is None:
37             continue
38
39         visited = {st for _, st in path}
40         for nxt in eff_domain(evicted, visited):
41             gn = g + 1
42             hn = heuristic(nxt, inv, eff_domain, free)
43             heapq.heappush(heap, (gn + hn, gn, (evicted, nxt),
44                                   path + [(evicted, nxt)]))
45
46     return None

```

Listing 2: Implementación A* en Python

7 Comparación de Métodos

Table 1: Comparación de estrategias de búsqueda para ejection chains.

Estrategia	¿Mín. eyectados?	Calidad solución	Memoria	Complejidad
BFS	Garantizado	Media	$O(b^d)$	$O(b^d)$
A* admisible	Garantizado*	Alta	$O(b^d)$	$O(b^d)$ reducido
DFS con límite	No garantiza	Variable	$O(d)$	$O(b^d)$
Greedy local	Subóptimo	Alta (local)	$O(1)$	$O(d)$
Beam Search	Aproximado	Alta	$O(w)$	$O(w \cdot d)$

*Óptimo garantizado cuando la heurística es admisible. b : factor de ramificación; d : profundidad; w : anch

Observación 7.1. Para grafos con dominios grandes ($|D(x)| \gg 1$) y soluciones GRASP densas, se recomienda **A*** con la heurística de la Definición 6.1 combinado con los tres niveles de poda del dominio efectivo. BFS es preferible cuando la heurística no aporta discriminación suficiente.

8 Procedimiento Completo

El Algoritmo 3 integra la construcción GRASP con la fase de mejora por ejection chains de longitud mínima.

Algorithm 3 GRASP + Ejection Chains con Conjunto Eyectado Mínimo

Require: Grafo bipartito G , función objetivo f_{obj} , umbral θ , profundidad máxima k_{max} , iteraciones I_{max}

Ensure: Mejor solución S^* encontrada

```

1:  $S^* \leftarrow \text{Ninguno}$ 
2: for  $i = 1, \dots, I_{\text{max}}$  do
3:    $S \leftarrow \text{GRASP\_Construir}(G, \alpha)$  ▷ Fase constructiva voraz-aleatoria
4:    $S \leftarrow \text{BúsquedaLocal}(S)$  ▷ Fase de búsqueda local estándar
5:   para cada par  $(x_0, y^*)$  candidato en  $S$  hacer
6:     Calcular  $D_0(x_0)$  según (1)
7:     Construir  $G_E$  con dominio efectivo (8), limitado a  $k_{\text{max}}$ 
8:      $\pi \leftarrow \text{A}^*(G_E, (x_0, y^*))$ 
9:     if  $\pi \neq \text{Ninguno}$  then
10:       $S' \leftarrow \text{AplicarCadena}(S, \pi)$ 
11:      if  $f_{\text{obj}}(S') > f_{\text{obj}}(S)$  then
12:         $S \leftarrow S'$ 
13:      end if
14:    end if
15:  end for
16:  if  $f_{\text{obj}}(S) > f_{\text{obj}}(S^*)$  then
17:     $S^* \leftarrow S$ 
18:  end if
19: end for return  $S^*$ 

```

9 Ejemplo Ilustrativo

Ejemplo 9.1. Sea $X = \{x_0, x_1, x_2\}$, $Y = \{a, b, c, d\}$, con asignación $f(x_0) = a$, $f(x_1) = b$, $f(x_2) = c$ y dominio $N(x_i) = Y$ para todo i .

Se desea mover $x_0 \rightarrow b$:

Paso	Acción	Estado
0	$D_0(x_0) = \{b, c, d\}$	x_0 quiere b
1	b está ocupado por x_1 ; eyectar x_1	cadena: (x_0, b)
2	$D_1(x_1) \cap D_0(x_0) = \{c, d\}$	
3	c está ocupado por x_2 ; elegir d (libre)	
	$x_1 \rightarrow d$ (libre ✓)	longitud = 1
<i>Reconstrucción:</i> $x_1 \rightarrow d$, $x_0 \rightarrow b$.		

La cadena tiene longitud 1 (un solo nodo eyectado), que es el óptimo para este caso.

10 Recomendaciones de Implementación

1. **Estructura de datos.** Mantener f^{-1} como diccionario inverso actualizable en $O(1)$ para calcular al eyectado en cada paso.
2. **Dominio efectivo como generador.** Implementar $D_{k,\text{eff}}(x_k)$ como un generador Python (lazy evaluation) para evitar materializar conjuntos grandes.
3. **Límite de profundidad $k_{\max}$.** Establecer $k_{\max} \leq |X|/4$ en la práctica; valores mayores rara vez aportan mejoras adicionales frente al costo computacional.
4. **Ordenamiento de la cola.** En BFS/A*, desempatar estados con igual $f = g+h$ priorizando $-\Delta f_{\text{obj}}$, lo que favorece cadenas simultáneamente cortas y de alta calidad.
5. **Caché de dominios.** Si los dominios no cambian entre iteraciones GRASP, almacenar $D_0(x)$ para cada x y reutilizarlos.
6. **Paralelismo.** Las búsquedas A* desde diferentes nodos iniciales (x_0, y^*) son independientes y pueden ejecutarse en paralelo con `concurrent.futures`.

11 Conclusiones

Se ha presentado una formulación rigurosa para la determinación de dominios en ejection chains sobre grafos bipartitos construidos mediante GRASP, y se ha modelado el problema de la cadena de longitud mínima como un problema de camino más corto en el grafo de eyección G_E .

Los principales aportes del documento son:

- La condición $D_k(x_k) \subseteq D_0(x_0)$ como mecanismo unificador del control de dominio.
- Tres niveles de poda (estructural, de calidad y anti-ciclo) que reducen el espacio de búsqueda sin comprometer la optimalidad.
- BFS como garantía de cadena mínima con costo $O(b^d)$.
- A* con heurística admisible como alternativa óptima con menor exploración efectiva.
- Un procedimiento completo GRASP + Ejection Chains (Algoritmo 3) listo para implementar.

References

- [1] F. Glover, “Ejection Chains, Reference Structures and Alternating Path Methods for Traveling Salesman Problems,” *Discrete Applied Mathematics*, vol. 65, pp. 223–253, 1991.
- [2] T. A. Feo y M. G. C. Resende, “Greedy Randomized Adaptive Search Procedures,” *Journal of Global Optimization*, vol. 6, pp. 109–133, 1995.
- [3] F. Glover y M. Laguna, *Tabu Search*. Kluwer Academic Publishers, 2000.
- [4] P. E. Hart, N. J. Nilsson y B. Raphael, “A Formal Basis for the Heuristic Determination of Minimum Cost Paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. SSC-4, no. 2, pp. 100–107, 1968.
- [5] M. G. C. Resende y C. C. Ribeiro, *Optimization by GRASP*. Springer, 2016.